

Assignment-6.3

2303A51242

Batch-05

Task-01:

Prompt:Generate a Python class named Student with attributes name, roll_number, and branch.

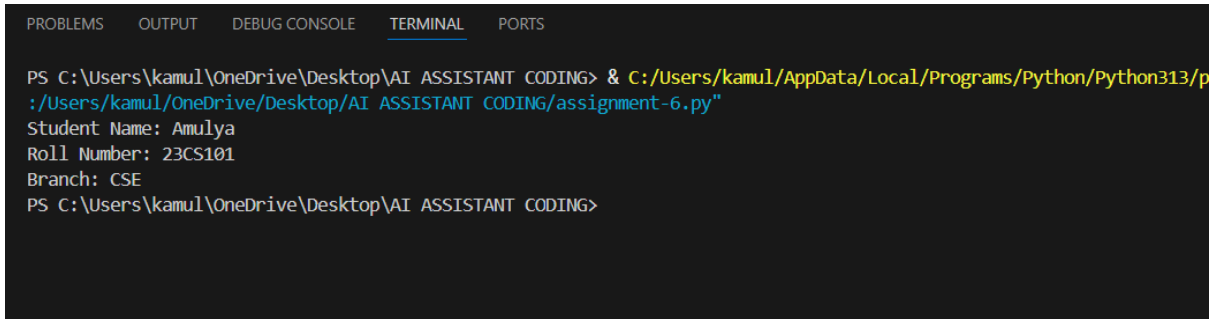
Include a constructor (`__init__`) and a method `display_details()` that prints all student information clearly. Create an object of the Student class with sample values and display the output on the console.

Analyze the generated Student class code for correctness, readability, and object-oriented design in simple terms.

Code:

```
assignment-6.py 2 ...
1  class Student:
2      def __init__(self, name, roll_no, branch):
3          self.name = name
4          self.roll_no = roll_no
5          self.branch = branch
6
7      def display_details(self):
8          print("Student Name:", self.name)
9          print("Roll Number:", self.roll_no)
10         print("Branch:", self.branch)
11
12
13 # Object creation
14 s1 = Student("Amulya", "23CS101", "CSE")
15 s1.display_details()
16
```

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/p
:/Users/kamul/OneDrive/Desktop/AI ASSISTANT CODING/assignment-6.py"
Student Name: Amulya
Roll Number: 23CS101
Branch: CSE
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation:

- AI correctly generated a class with `__init__()` constructor.
- Attributes are clearly defined and readable.
- `display_details()` method prints all details correctly.
- Code follows object-oriented principles and is easy to understand.

Task-02:

Prompt: Write a Python function that prints the first 10 multiples of a given number using a for loop. Explain how the for loop works in the above code and why it is suitable for this task. Generate the same functionality to print the first 10 multiples of a number using a while loop instead of a for loop. Compare the for loop and while loop implementations in terms of readability and usage.

Code:

```
assignment-1.py assignment-3.py assignment-6.py
assignment-6.py > ...
1 def print_multiples(num):
2     for i in range(1, 11):
3         print(num * i)
4
5 print_multiples(5)
6
7 def print_multiples_while(num):
8     i = 1
9     while i <= 10:
10        print(num * i)
11        i += 1
12
13 print_multiples_while(5)
14
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [ ] [ ] ... | [ ] [ ] X
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/kamul/OneDrive/Desktop/AI ASSISTANT CODING/assignment-6.py"
5
10
15
20
25
30
35
40
45
50
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/kamul/OneDrive/Desktop/AI ASSISTANT CODING/assignment-6.py"
5
10
15
20
25
30
35
40
45
50
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation:

- for loop is concise and ideal when iteration count is known.

- while loop provides more control but requires manual increment.
- Both approaches produce correct results.

Task-03:

Prompt: Write a Python function that classifies a person's age into Child, Teenager, Adult, or Senior using nested if-elif-else conditions. Explain how each condition works in the age classification function. Rewrite the age classification program using a simplified or alternative conditional structure such as ranges or dictionary-based logic.

Code:

```
assignment-6.py > classify_age
1 def classify_age(age):
2     if age < 13:
3         return "Child"
4     elif age < 20:
5         return "Teenager"
6     elif age < 60:
7         return "Adult"
8     else:
9         return "Senior"
10
11 print(classify_age(25))
12 def classify_age_simple(age):
13     categories = [
14         (13, "Child"),
15         (20, "Teenager"),
16         (60, "Adult")
17     ]
18     for limit, label in categories:
19         if age < limit:
20             return label
21     return "Senior"
22 print(classify_age_simple(25))
23
```

Output:



The screenshot shows a Windows terminal window with the following tabs: PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The terminal displays the following commands and output:

```
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/kamul/OneDrive/Desktop/AI ASSISTANT CODING/assignment-6.py"
Adult
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/kamul/OneDrive/Desktop/AI ASSISTANT CODING/assignment-6.py"
Adult
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation:

- Conditions are logically ordered from lowest to highest.
- Alternative approach improves readability and scalability.
- AI-generated logic is correct and efficient.

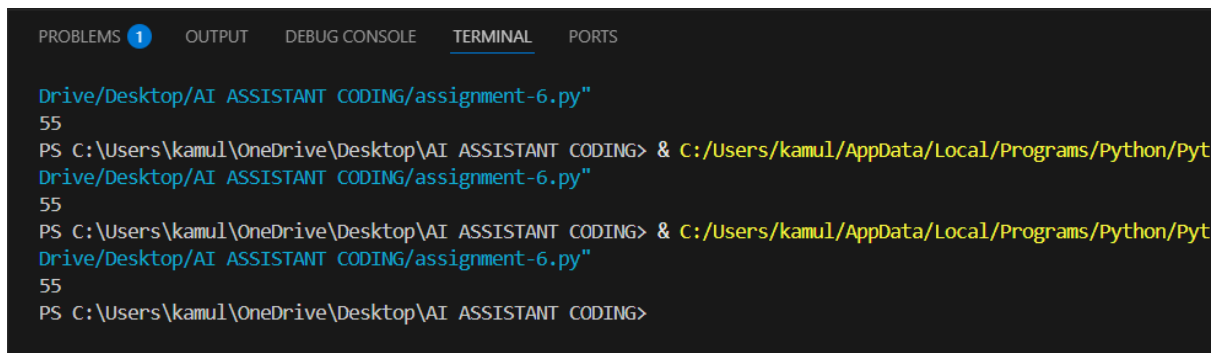
Task-04:

Prompt: Generate a Python function `sum_to_n(n)` that calculates the sum of the first `n` natural numbers using a for loop. Analyze the generated for loop code and explain how the sum is calculated step by step. Provide an alternative implementation of `sum_to_n(n)` using a while loop or a mathematical formula and compare it with the for loop approach.

Code:

```
assignment-6.py > ...
1  def sum_to_n(n):
2      total = 0
3      for i in range(1, n + 1):
4          total += i
5      return total
6
7  print(sum_to_n(10)) ==
8  def sum_to_n_while(n):
9      total = 0
10     i = 1
11     while i <= n:
12         total += i
13         i += 1
14     return total
15  print(sum_to_n_while(10))
16  def sum_formula(n):
17      return n * (n + 1) // 2
18  print(sum_formula(10))
19
20
```

Output:



The screenshot shows a VS Code terminal window with the 'TERMINAL' tab selected. The terminal displays the execution of a Python script named 'assignment-6.py' located at 'Drive/Desktop/AI ASSISTANT CODING/assignment-6.py'. The command prompt shows the user running the script using the Python interpreter. The output of the script is '55'. The terminal window has a dark background with light-colored text.

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
Drive/Desktop/AI ASSISTANT CODING/assignment-6.py"
55
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Pyt
Drive/Desktop/AI ASSISTANT CODING/assignment-6.py"
55
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Pyt
Drive/Desktop/AI ASSISTANT CODING/assignment-6.py"
55
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation:

- Loops are easy to understand for beginners.
- Formula method is most efficient ($O(1)$ time complexity).
- AI suggested all valid and correct approaches.

Task-05:

Prompt: Create a Python class named BankAccount with attributes for account holder name and balance.

Include methods deposit(), withdraw(), and check_balance(). Show sample usage of the BankAccount class by performing deposit and withdrawal operations and displaying the updated balance. Add meaningful comments to the BankAccount class and explain how each method works in simple language.

Code:

```
assignment-6.py > BankAccount > withdraw
1 class BankAccount:
2     def __init__(self, account_holder, balance=0):
3         self.account_holder = account_holder
4         self.balance = balance
5     def deposit(self, amount):
6         self.balance += amount
7         print("Deposited:", amount)
8     def withdraw(self, amount):
9         if amount <= self.balance:
10             self.balance -= amount
11             print("Withdrawn:", amount)
12         else:
13             print("Insufficient balance")
14     def check_balance(self):
15         print("Current Balance:", self.balance)
16 # Demonstration
17 acc = BankAccount("Amulya", 5000)
18 acc.deposit(2000)
19 acc.withdraw(3000)
20 acc.check_balance()
21 acc.withdraw(5000)
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - [ ] [X] ... | [ ] [X] X
```

```
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING> & C:/Users/kamul/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/kamul/OneDrive/Desktop/AI ASSISTANT CODING/assignment-6.py"  
Deposited: 2000  
Withdrawn: 3000  
Current Balance: 4000  
Insufficient balance  
PS C:\Users\kamul\OneDrive\Desktop\AI ASSISTANT CODING>
```

Explanation:

- AI-generated class structure is well-organized.
- Methods correctly update and validate balance.
- Meaningful method names improve clarity.
- Code is reusable and easy to maintain.

